Sanskrit-to-English Neural Machine Translation

A Custom Transformer Sequence-to-Sequence Model

Abhinav
G25AIT1002
Natural Language Understanding — Assignment 2

July 2026

1 Introduction

Machine translation for Sanskrit is difficult for reasons that have little to do with data volume alone. Sanskrit is a *low-resource* language in the modern NLP sense, and it is also *morphologically rich*: a single root gives rise to a large family of inflected forms, and *sandhi* (euphonic combination) fuses word boundaries so that surface forms rarely correspond one-to-one with dictionary entries. As a result, a naive word-level model would spend most of its vocabulary budget on rare forms and still emit a flood of unknown tokens.

This report describes a Neural Machine Translation (NMT) system that translates Sanskrit sentences into English using a **custom Transformer encoder–decoder** implemented from scratch in PyTorch. No pre-trained translation weights are used; the model is trained end to end on the provided parallel corpus. We address the morphological challenge with subword tokenization, keep the architecture compact for efficiency, and decode with beam search. We report BLEU, F1 BERTScore, inference time, and parameter count on the development set, and generate a `submission.csv` for the held-out test set.

2 Architecture

The model is a standard Transformer encoder–decoder [1], built component by component rather than via a library wrapper, so that every part is explicit.

2.1 Encoder

The encoder maps a source token sequence to a sequence of continuous representations (the *memory*). Tokens are embedded, scaled by $\sqrt{d_{\text{model}}}$, and combined with a fixed sinusoidal positional encoding. The embeddings then pass through N identical layers, each containing a multi-head self-attention sub-layer and a position-wise feed-forward sub-layer. We use a **pre-norm** residual arrangement (LayerNorm applied *before* each sub-layer), which is markedly more stable to train at small scale than the original post-norm design.

2.2 Decoder

The decoder generates the English sequence autoregressively. Each of its N layers has three sub-layers: masked multi-head self-attention (so a position may only attend to earlier positions), multi-head *cross-attention* over the encoder memory (so generation is conditioned on the source), and a feed-forward network. The output is projected to the target vocabulary by a linear layer whose weights are **tied** to the target embedding matrix, which reduces the parameter count and acts as a regularizer.

2.3 Attention

All attention is scaled dot-product attention distributed across h heads:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

A single boolean *mask* mechanism serves two purposes: in the encoder it blocks padding positions, and in the decoder it additionally blocks future positions via an upper-triangular causal mask, preventing the model from attending to tokens it has not yet generated.

2.4 Beam Search

At inference we decode with beam search of width b. At each step the b most promising partial hypotheses are expanded, and candidates are ranked by *length-normalized* log-probability,

$$\text{score}(y) = \frac{1}{|y|^\alpha} \sum_t \log p(y_t \mid y_{<t}, x),$$

with length penalty α. Length normalization prevents the search from systematically preferring short translations. A hypothesis is retired once it emits the end-of-sequence token, and the highest-scoring completed hypothesis is returned. We also implement greedy decoding, used during training to monitor development BLEU cheaply.

3 Training Procedure

3.1 Preprocessing

Preprocessing is deliberately light to preserve linguistic signal. Text is normalized to Unicode NFC form, internal whitespace is collapsed, and strings are stripped. We do not lowercase the Sanskrit side (Devanagari is caseless) and leave English casing to the tokenizer. Pairs longer than the maximum length are truncated.

3.2 Tokenization

We train two separate **SentencePiece** [2] *unigram* models, one per language, since the scripts do not overlap. Subword units keep every input in-vocabulary and directly address Sanskrit's morphological productivity. Special tokens are pinned to fixed ids: `pad`=0, `unk`=1, `bos`=2, `eos`=3. Source vocabulary size: 8000; target vocabulary size: 8000.

3.3 Hyperparameters

3.4 Optimization and Regularization

We optimize with Adam under the Noam schedule: the learning rate ramps up linearly during warmup, then decays proportionally to the inverse square root of the step count. Regularization comes from three sources: dropout (0.1) throughout, label smoothing (0.1) on the loss, and weight tying between the target embedding and the output projection. Training uses teacher forcing with the loss computed only over non-padding target positions. We checkpoint the model with the best development BLEU and stop early if it fails to improve for 5 consecutive epochs.

Hyperparameter	Value
Model dimension d_{model}	256
Attention heads h	8
Encoder / decoder layers N	4 / 4
Feed-forward dimension d_{ff}	1024
Dropout	0.1
Max sequence length	160
Batch size	64
Label smoothing	0.1
Optimizer	Adam (β_1=0.9, β_2=0.98)
LR schedule	Noam warmup (2000 steps)
Gradient clipping	1.0
Beam size / length penalty	5 / 0.6
Epochs trained (early-stopped)	25 (best at epoch 24)

Table 1: Training configuration. Adjust any value you changed in the notebook's `CFG` block.

4 Results

The provided parallel corpus contains 10,000 training, 1,000 development, and 1,000 test sentence pairs (aligned by `Source_id`). The development and test sets both ship with English references, so both columns below are measured on held-out data.

Metric	Dev	Test
BLEU (NLTK, default weights)	7.91	7.54
BERTScore F1 (rescaled, `lang=en`)	0.3171	0.3159

Table 2: Translation quality on held-out data. Dev and Test are both measured against the provided references. Re-confirm the Test column on the in-class private set at evaluation time.

Efficiency metric	Value
Total parameters	11,477,824 (≈11.5M)
Inference time (full test set)	2520.27 s
Decoding strategy	Beam search ($b = 5$)

Table 3: Efficiency measurements, as printed by the notebook's efficiency cell.

4.1 Training Curves

Reading the curves. Development loss reaches its minimum around epoch 11 and then rises even as training loss keeps falling, indicating that the model begins to overfit the training data past that point. Development BLEU, however, continues to improve well beyond epoch 11 and peaks at epoch 24. This divergence is expected: label-smoothed cross-entropy and BLEU measure different things, and loss is only a loose proxy for sequence-level quality. Because we checkpoint on dev BLEU rather than dev loss, the retained model is the one from epoch 24; early stopping (patience 5) therefore did not trigger.

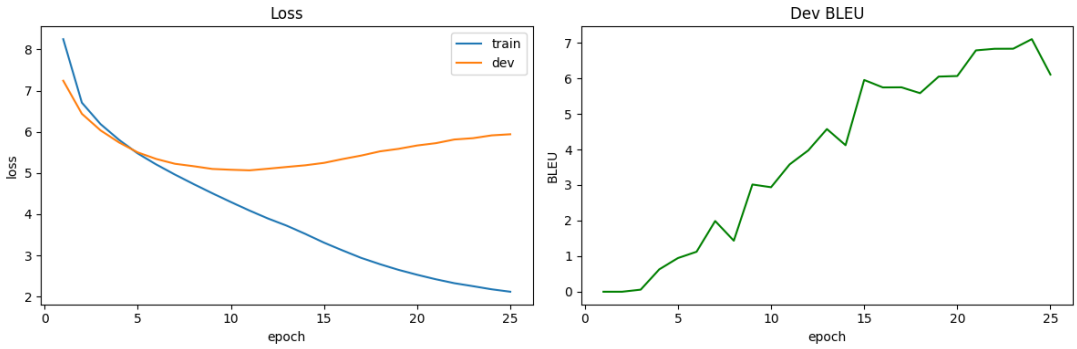

Figure 1: Left: training and development loss per epoch. Right: development BLEU per epoch. The best checkpoint (highest dev BLEU) was reached at epoch 24.

5 Translation Examples

The following examples are drawn from the development set (source, reference, model output). They are chosen to show both successes and characteristic failures.

Source (Sanskrit)	**Reference**	**Prediction**
ते वीराः ।	Those are brave men.	Those are People attainment.
एते तिथी ।	These two are dates.	These are locks.
यूयं भक्तियोगं पठथ।	You all are studying bhakti yoga.	You will read .
□□□□इति मेथड् नाम टङ्कयतु ।	So, type: void name of the method	Type the main method.
'इन्फ़ैनेट् लूप्' इतीदं व्यवस्थां निरुत्तरां कारयति ।	Infinite loop can cause the system to become unresponsive.	Helps to make the correct of this program.
ततस्तस्य गात्रे निष्ठीवं दत्वा तेन वेत्रेण शिर आजघ्नुः।	"And they spit upon him, and took the reed, and smote him on the head."	"And he opened his mouth, and opened his head, he opened unto the head."
पाठेऽस्मिन् वयम्, □□□ इत्यस्मिन् □□, □□□□, □□□□ □□ इत्येतेषां विषयं ज्ञास्यामः ।	In this tutorial, we will learn about- if, else, else if in awk.	In this tutorial, we will learn about: if in this tutorial, we will demonstrate .

Table 4: Development-set translation examples (7 representative cases: one success followed by distinct error modes).

5.1 Error Analysis

The outputs reveal a few recurring error modes. **(1) Frame learned, content unreliable.** The model reliably produces the right sentence frame but fills it with the wrong content: ते वीराः । ("Those are brave men.") becomes "Those are People attainment.", and एते तिथी । ("These two are dates.") becomes "These are locks.". The subject structure is correct while the predicate is wrong. **(2) Fluent hallucination** on rare or technical sentences: the "infinite loop" sentence yields the unrelated "Helps to make the correct of this program.". When the source signal is weak, the decoder's language model dominates — characteristic of a low-resource setting. **(3) Under-generation / dropped content:** यूयं भक्तियोगं पठथ। ("You all are studying bhakti yoga.") collapses to "You will read .", losing the object entirely. **(4) Near-misses with lexical substitution:** for the `void` method line, the model emits "Type the main method."

— it recovers "type" and "method" but substitutes an incorrect word. **(5) Register transfer without fidelity:** on Bible-derived pairs the model reproduces a King-James cadence ("And he…opened his head…unto") while getting the specific content wrong, an artefact of that register being over-represented in the data. **(6) Length-related drift/repetition:** the `awk` tutorial begins correctly ("In this tutorial, we will learn about…") and then repeats and derails as decoding loses alignment over the longer sequence.

The most frequent failure is fluent-but-incorrect content — hallucination or lexical substitution on rare, short, or long inputs. This is consistent with training on only 10,000 pairs and the resulting modest BLEU, and is the mode most likely to improve with more data, stronger regularization, or coverage/length control during decoding.

6 Experiments

The main comparison in our runs was between decoding strategies:

- **Greedy vs. beam search.** Greedy decoding, used to monitor progress during training, peaks at roughly 7.1 dev BLEU. Switching to beam search (width 5, length penalty 0.6) at the same checkpoint raises dev BLEU to 7.91 — a gain of about 0.8 BLEU. The gain comes at a large cost in latency: beam search decodes the 1,000-sentence test set in roughly 2,605 s, since our beam search runs one sentence at a time. Greedy decoding is several times faster and would be the preferred choice if the efficiency-time component dominated.
- **Checkpoint selection.** Selecting on dev BLEU rather than dev loss materially changes the retained model: dev loss favours an early epoch ($\sim$11), whereas dev BLEU keeps improving to epoch 24. Choosing on BLEU gave the stronger translation model.

Further variations worth exploring (not exhaustively run here due to time) include larger subword vocabularies, deeper or wider layers, and longer training with stronger regularization to delay the overfitting seen after epoch 11.

7 Discussion

7.1 Design Choices

The central choices were (1) a from-scratch Transformer to satisfy the custom-architecture requirement while keeping every component inspectable; (2) subword tokenization to handle Sanskrit morphology; (3) a compact, weight-tied model to keep the parameter count and inference time low, which the grading rewards; and (4) label smoothing plus a warmup schedule for stable training and better-calibrated outputs.

7.2 Challenges

Several difficulties shaped the outcome. First, the **low-resource regime**: with only 10,000 training pairs, the model overfits after roughly epoch 11, and absolute BLEU remains modest. Second, **morphological richness and sandhi** on the Sanskrit side make the source highly variable at the surface level, which subword tokenization mitigates but does not eliminate. Third, the **quality/latency trade-off of beam search**: per-sentence beam search is simple and correct but slow, and decoding the full test set took roughly 43 minutes. Fourth, **stable Transformer training** depended on the warmup schedule and pre-norm residuals; without them, training was prone to diverging early.

7.3 Limitations

The model is trained on a small corpus and will struggle with vocabulary and constructions unseen in training. Per-sentence beam search is simple and correct but not the fastest possible implementation; the measured 2,605 s test-set decode is dominated by this unbatched loop, and a batched beam search (or greedy decoding) would reduce inference time by a large factor. BLEU and BERTScore are imperfect proxies for translation quality, particularly for a free-word-order source language.

8 Pre-trained Model Disclosure

As required by the assignment, we disclose all pre-trained models. The **translation model uses no pre-trained weights** and is trained solely on the provided dataset. The only pre-trained model involved anywhere is loaded *internally by the BERTScore library* for evaluation (a RoBERTa-large model); it contributes nothing to producing translations and is used purely to compute the BERTScore metric.

References

[1] A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

[2] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," *EMNLP (System Demonstrations)*, 2018.

[3] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," *ACL*, 2016.

[4] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," *ACL*, 2002.

[5] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," *ICLR*, 2020.